

Section A: Conceptual Understanding

1. Java Runtime and Platform Independence

The Java runtime enables a wallet application to work across different operating systems through the "Write Once, Run Anywhere" (WORA) principle. When Java code is compiled, it is converted into Bytecode, which is platform-neutral. The Java Virtual Machine (JVM) installed on a specific OS (Windows, macOS, Linux) translates this Bytecode into machine-specific instructions at runtime.

2. Precision Handling for Money Transfers

In financial applications, using floating-point types like `float` or `double` is avoided because they can lead to rounding errors due to binary representation. Instead, Java uses the `BigDecimal` class. It provides absolute precision for decimal arithmetic, allowing developers to control rounding behavior (e.g., `setScale(2, RoundingMode.DOWN)`) to ensure no money is "lost" during transfers.

3. Transfer Validation Logic

Transfer validation involves a series of logical checks to ensure a transaction is secure and valid before execution:

- **Balance Check:** Verifying the sender has sufficient funds.
- **Identity Verification:** Ensuring the sender and receiver accounts are active and valid.
- **Limit Validation:** Checking if the amount exceeds daily or per-transaction limits.
- **Input Sanitization:** Ensuring the transfer amount is a positive non-zero value.

4. Loop Usage for Monthly Summaries

Loops (such as `for` or `forEach`) are used to iterate through a list of transactions stored for a specific month. As the loop progresses, it can:

- **Filter** records based on a date range.
- **Accumulate** totals for credits and debits.
- **Format** each transaction line for a readable summary report or UI display.

5. Collections for Transaction Storage

The Java Collections Framework (e.g., `ArrayList`, `LinkedList`, `HashSet`, or `HashMap`) is ideal for transaction storage because:

- **Dynamic Resizing:** Unlike arrays, collections grow automatically as more transactions are added.
- **Built-in Methods:** They offer easy ways to sort transactions by date, search for specific IDs, or clear history.
- **Type Safety:** Using Generics (e.g., `T`) ensures that only transaction objects are stored, preventing runtime errors.

6. Payment Modes using Polymorphism

Polymorphism allows the wallet to treat different payment methods (Credit Card, UPI, Bank Transfer) as a single generic type, such as a interface.

- **Implementation:** Each specific mode implements a method.
- **Benefit:** The main application doesn't need to know the details of *how* a payment is processed; it simply calls `pay()`. This makes it easy to add new payment methods in the future without changing the existing core logic.